[Vision Paper] Towards AI-Conducted Benchmarking: Autonomous Design and Interpretation of Performance Experiments on Data Systems

Patrick K. Erdelt
Berliner Hochschule fuer Technik (BHT)
Berlin, Germany
patrick.erdelt@bht-berlin.de

Abstract

Large language model agents already write SQL, tune configuration knobs, and diagnose incidents in cloud environments, yet they cannot conduct a benchmark study: select instruments, design a valid comparison, judge validity, and attribute effects. We argue this reflects an interface gap, not a capability gap, and advance a thesis with reach beyond benchmarking: a general reasoning agent conducts valid experiments in a domain when the infrastructure formalizes the domain's tacit expertise as a structured, machine-actionable interface. We make the case in performance benchmarking of data systems, the empirical backbone of systems research, where a wrong result is costly and often silent. The enabler is a two-sided contract between agent and benchmarking infrastructure: on the input side, a catalog of workloads *and* of systems under test that exposes intent, parameters, and the configuration choices a fair comparison depends on; on the output side, a self-describing result folder bundling metrics, provenance, and validity indicators. We describe a prototype realization on a Kubernetes-based benchmarking framework, built to close the experiment loop for a DBMS comparison study, and outline a research agenda spanning contract standardization, guardrails against invalid experiments, and what reproducibility requires once experiments are designed by a model.

Keywords

Benchmarking, Kubernetes, DBMS, LLM Agents, Reproducibility, Experiment Orchestration

1 Introduction

> *Can a general reasoning agent conduct valid experiments in a domain it was never trained to practice? And if so, what must the domain's infrastructure expose to let it?*

Our answer is that the barrier is not the agent's intelligence but the infrastructure's interface. Expertise that looks like judgment is, in substantial part, knowledge left tacit in tools built for human operators. Expose that knowledge as structure a reasoner can act on, and a general agent behaves as a domain expert. We call an interface with this property *AI-ready*, and we make the case in performance benchmarking of data systems, where a wrong result is often silent, making the claim harder to uphold than elsewhere.

Two obvious paths promise the same goal, and we take neither. One is to wait for a large enough model that has read everything. But a model that has read everything has also read a great deal of conflicting and outdated material: superseded advice, marketing claims, benchmarks run wrong, contradictory rules of thumb. It cannot tell us which of these it will draw on for a given experiment, and neither can we. Benchmarking needs not more knowledge in the model but a predictable account of what governs each decision, and unbounded training gives the opposite. The other path is to train or fine-tune a specialist model. But its weights cannot be inspected, versioned, or separated from what they memorized, and a study whose method lives in weights is not one anyone can audit or reproduce. A fine-tuned model is also frozen at training time, blind to this cluster and to a system released last week. We propose instead to put the knowledge in the interface, not the model. The contract states what governs each choice, so the agent's decisions are predictable and inspectable, a general model suffices, and the system stays current whichever model reads it.

Conducting a benchmark study is the sharp test of that claim, because it demands knowledge no benchmark ships with. A study is a reasoned sequence. Select the instrument that fits the question, and know its non-scope. Set it up as a factorial design, with repetitions and tuning parity. Run it. Judge validity against artifacts like warm-versus-cold caching. Weigh the results against their statistical variance. And often design a discriminating follow-up, to separate the explanations the first results leave open. Each step draws on knowledge held by experts and left out of the tools they use, and mastering all of them at once remains the open challenge.

Its output is not a number but a *result* in the full sense: the measurements, every parameter that shaped them across host, system under test, and workload, *and* the traces of how the system behaved. Parameters say what was asked; traces say how the system responded, and attribution lives in their correlation. That presupposes a tool that knows what a buffer pool or a planner is *for*: one that behaves as if it understands the system it measures, not merely records its outputs. A throughput figure without this context is neither reproducible nor comparable, let alone explicable.

Both setup and interpretation are expert craft today. Orchestration frameworks automate the *execution* of studies [10], but their workloads are parameterized from folklore and their results read from logs whose meaning lives in expert heads. LLM agents write SQL, tune knobs [21, 22], and diagnose incidents in Kubernetes clusters [5], but cannot yet *conduct an experiment*. This is an interface gap of the infrastructure, not a capability gap of the agents: benchmarking is not AI-ready. Kubernetes is where this matters most, since its flexibility — placement, co-location, storage class, resource limits — turns performance into a question of configuration no static benchmark captures.

EDBT '27, Lille (France)

The enabler is a deliberately small, two-sided contract. A catalog of workloads and systems, describing not just their parameters but what a system's components are for, together with a generated environment descriptor, lets the agent express an experiment as a validated specification; a result contract turns the summary into a navigable, self-describing entry point for a bounded-context reader. To an engineer this means declaring a few systems and receiving a study rigorous by construction; to a scientist, designing a new benchmark without the labor of testbeds and multi-dimensional sweeps. The contract hands each the part they lack and asks only the judgement no tool can supply.

Benchmarking then changes in kind, from an occasional human-triggered campaign to a standing capability in which experiments answer events. A new release lands overnight and by morning an agent has compared it against the archive and isolated a regression's cause; a published study, archived as specification and contract, is re-run by a reviewer's agent on another cluster, turning reproducibility from a badge into an executable check. These are the *near* end of a longer road whose horizon is the opening question, open-ended and cross-layer, and beyond it questions that turn benchmarking into decision-making under a budget. Our agenda charts what lies between.

We contribute the vision and its use cases, the two-sided contract as its enabling abstraction, a prototype realization on an existing orchestration framework [12] that closes the loop for a comparison study, and a research agenda for the trust such an experimenter must earn and the reach it gains.

2 Related Work

Four research lines converge on our vision; for each, we state its ambition, its state of the art, and what it leaves open — the gap paragraph at the end of this section collects why their combination is both missing and necessary.

2.1 Reproducible Benchmarking of DBMS in the Cloud

The vision of this field: any performance claim can be re-established by anyone, anywhere — the experiment as a portable, repeatable artifact rather than expert craft.

The TPC family standardizes *what* to measure, extensible harnesses provide the *how* [8], and orchestration frameworks run it at scale and in Kubernetes [9–12, 15, 33], under SPEC's methodological principles [29]. The pattern is broad: a full cycle as a Kubernetes operator [18], declarative specifications [6, 19], and commercial services [32]. Yet the craft resists automation: a single benchmark demands deep knowledge [4], experts fall into documented pitfalls [13, 30], and comparing systems is itself subject to systematic review [34].

Execution is thus repeatable; the experimenter is not. Even determining a valid setup can take a hand-run sequence of experiments its authors note "could also be automated" [27]. Closest to our input side, the testing environment has been modeled as a *variability model* over data, hardware, and DBMS parameters [28], which we extend from a human-authored aid into an agent-navigable contract. Each framework runs the experiment it is *given*; ours exists so an agent can decide what to give it.

2.2 LLM Agents for Database Tuning

This line pursues the self-managing database, ending expert administration as the price of good performance. *Knob tuning* runs from learned advisors [1] to LLMs that mine manuals [35], refine search spaces [21], tune from prompts [14], and map workloads to configurations [16]. *Agentic administration* adds agents that emulate a DBA [22], mine source code [40], diagnose anomalies [42], and run production maintenance in production [41], while *code synthesis* generates whole engines and estimators [36, 37]. Closest to us, a multi-agent tuner adds a performance digest and fail-closed checks [24], and database gyms serve as training-data factories for self-driving components [23].

All share one interface shape: scalar optimization of a single system, through an *implicit* input contract and a result channel reduced to a reward. There is no provenance, placement, repetition, or validity, because the benchmark is trusted as an oracle and an invalid run is noise, not a finding. This cannot express an experiment: no factorial design, no cross-system comparison, no trust judgment. We invert the benchmark's role, from disposable oracle to object of design and interpretation, and replace the scalar interface with an explicit, versioned contract these systems could adopt as substrate — for as they become synthesizable in hours, evaluation becomes the bottleneck.

2.3 Agentic Environments for Cloud Operations

The ambition here is the autonomous cloud: infrastructure that detects, diagnoses, and repairs its own incidents.

AIOpsLab and ITBench curate a boundary between a probabilistic agent and a cluster [5, 17], the closest architectural relatives of our contract. Their purpose is inverted: the environment is instrumented to *evaluate the agent*, whereas we instrument benchmarking so the agent can *evaluate a system*. The inversion changes what the boundary must express — experimental-design vocabulary and validity metadata, not incident context and remediation — and the reported difficulty is interpreting the telemetry given [5], exactly the problem our result side addresses.

2.4 Autonomous Experimentation and Machine-Actionable Results

The broadest vision is science at machine speed: discovery limited by ideas, not by the labor of experimentation.

General agents automate the hypothesis–experiment–analysis loop [25, 31]; data-science agents conduct its *observational* form but still fail at end-to-end orchestration [20]. The Lab Agent Protocol standardizes the agent-to-instrument edge with typed, uncertainty-bearing results [43], and machine-actionable metadata has matured for data [2, 26, 38]. Closest, DIAMetrics extracts benchmarks from production workloads [7] and others audit whether benchmark artifacts are correct [39].

None of this covers systems performance experiments, with their factorial configurations, per-run metrics, and validity checks. Our agent is the experimental counterpart to the data agents: it does not analyze data that exists but designs interventions that create it. The interpret phase of our loop is itself a data-agent task over the result folder, and the Lab Agent Protocol is structurally our descriptor and result contract for instruments instead of clusters — components of the vision, not competitors. DIAMetrics and the auditing work build or verify benchmarks, as we do, but never hand the result to an autonomous experimenter.

Research gap and contribution. The volume of adjacent work is not evidence this vision is taken but that the field keeps reaching for it blindly: every system above had to run a benchmark and read its result, and each built a private, throwaway way to do

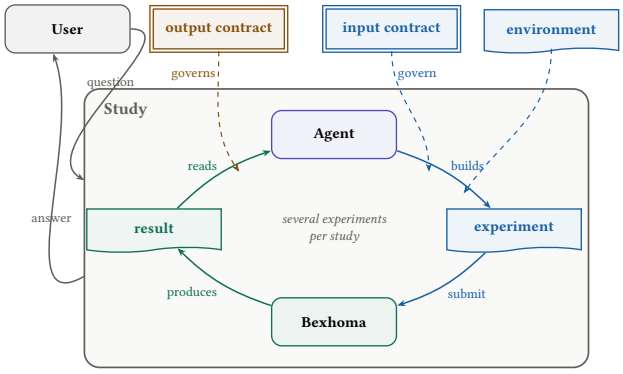

Figure 1: A *study* answers one question and may take several experiments to do so. The user sends a question to the study and receives an answer; inside, the agent cycles: it builds an experiment from the input contract and environment descriptor, Bexhoma runs it, and the agent reads the result through the output contract, iterating until the question is answered. Contracts are double-ruled, artifacts torn-page, processes rounded.

so [6, 24] — instancing the same interface without naming it. No one has treated the boundary between agent and benchmarking infrastructure as an object worth building once. That recognition is our contribution: a two-sided contract — catalog of workloads and systems in, self-describing result folder out — that turns the throwaway harness into a standing one and the agent from a tuner into an experimenter. The dense prior work clusters entirely on the optimization side of that line; past it we find open ground.

3 The Two-Sided Contract

By a *contract* we mean a versioned, machine-readable agreement between agent and benchmarking infrastructure that fixes what each side may express and what it may assume. Three properties separate it from documentation. It is *enforced*: validation rejects what the contract does not cover, and generation guarantees that what it promises is present — which is also what turns the with-/without comparison into a measurable variable rather than a preference. It is *versioned* and archived with every experiment, so agent behaviour is auditable against the contract it actually saw, unlike knowledge held in model weights, which is unversioned, unauditable, and cluster-blind. And it is *inspectable* by humans and machines alike. The term is converging on us from elsewhere: engine synthesis binds a workload to a generated system through what its authors likewise call a contract [36], and agent tooling there is similarly defined as a closed set of tools that forms the entire agent–system boundary. Their contract binds a workload to a synthesized system; ours binds an experimenter to a benchmarking infrastructure. The hard part of such a contract is not its syntax but its content. What an instrument measures and what it cannot, which parameter combinations are meaningful, what invalidates a run, which result is implausible — this is decades of accumulated community knowledge, the reason benchmark creation has been called an art [3]: our field has long written down its lessons for human readers [4, 13, 29, 30]. Restating that corpus in a form machines can act on is a community-scale program, comparable to what the standardization of workload definitions once was. This paper contributes the frame and a seeded catalog, not the finished corpus.

Figure 1 shows a study and the cycle within it; Figures 2–4 then illustrate each artifact end to end, and the following subsections state each artifact and its objective.

3.1 The Input Side: The Agent as Planner

Expresses an experiment: selecting a workload and its scope, configuring the systems under test, declaring the factors to vary, as a specification the infrastructure validates before running.

Workload and system catalog. The catalog spans two independent axes: *what* to run and *what to run it on*. The workload axis lets the agent map a goal-level question to the right instrument by stating *intent* before parameters — why a workload exists, when to choose it, its non-scope — so selection is reasoned, not guessed (Figure 2). The system axis spans a spectrum of configurability, from a managed service exposing only an endpoint to a fully specified deployment with pinned version, knobs, physical design, and resource limits. This spectrum is where the confounders of a fair comparison live, tuning parity above all, and the contract turns each into a recorded field rather than a tacit decision. One such decision the contract makes by default: systems under test are benchmarked one at a time, never side by side, so a measurement records the declared factor rather than the interference of co-located systems contending for the same node. An experiment references one entry of each kind. The environment descriptor completes the input side: generated from the cluster at experiment time, it states node groups, resource limits, and storage classes with their benchmarking-relevant semantics (a network-attached class implies latency sensitivity), so the agent designs against the hardware it actually has rather than assumptions from training.

Experiment specification. The specification is the interchange format of the loop: written by the agent from catalog and descriptor, checked by validation, stored with the results as provenance (Figure 3). It records not only *what* to run but *why*, carrying a hypothesis and the rival it means to discriminate, so the trajectory becomes a readable chain of reasoning and every later claim points to the spec that tested it.

Validation. A dry-run mode checks a specification against workload semantics and the descriptor before any cluster resource is touched, rejecting with structured errors the agent can self-correct from. It fails closed on unknown fields, since a silently ignored key yields an experiment that looks valid but is not, and it returns an estimated run count so a factorial design that would explode the budget is caught as economically invalid before it runs.

3.2 The Output Side: The Agent as Reader

Reads a self-describing result folder and returns a bounded answer: a verdict tied to the hypothesis, each claim linked to evidence, its own validity and scope stated.

The answer restates the hypothesis and its rival, gives a conclusion scoped to what was tested, links each claim to the evidence in the folder, notes which validity checks passed or were skipped, and proposes a follow-up for what the result left open, linked to the run it extends so a study's lineage is recorded structurally rather than left in prose. This is the output side of pre-registration: a verdict tied to the question it began with, and to evidence a

```yaml
# catalog.yml (excerpt) -- contract v0.1
# two axes: WHAT to run, WHAT to run it on
workloads:
  tpch:
    why:  analytical join & aggregation,
          22 queries
    when: join-heavy or reporting tasks;
          NOT point reads or updates
    cost: load ~10 min per sf and system
    supports: [PostgreSQL, PgDuckDB]
    params:
      active_queries:
        type: subset of [1..22]
        semantics: 5,7,8,9,21 = multi-way
          joins; 1,6 = scan-dominated
      scaling_factor: {type: int, unit: GB}
    rounds:
      type: list[int]   # parallel-client sweep
      semantics: one benchmarking round per
        entry; [1,4] = single then 4 streams
    produces:
      per_query: {metric: latency, unit: ms}
      summary: [Power@Size, Throughput@Size]
  ycsb:
    why:  key-value CRUD, tunable r/w mix
    when: raw KV throughput; NOT joins
    # ... omitted ...
systems:
  DatabaseService:      # one end: nothing
    endpoint: my-svc:5432   #  to configure
  PostgreSQL:           # the other end
    image: postgres:18.3
    knobs:
      shared_buffers: {type: memory}
      random_page_cost:
        type: float
        status: reference-only   # in template
      # ... omitted ...
    physical_design: {indexes: true,
      statistics: true}   # capability
    profiles:
      analytical-ssd:
        why: OLAP on node-local NVMe,
          sized from the memory limit
        requires: {storage_class: [ssd, null]}
        derive:
          shared_buffers: 0.3125*memory_limit
          effective_cache_size:
            0.75*memory_limit
  PgDuckDB:
    extends: PostgreSQL   # inherits knobs
    profiles: {analytical-ssd:
      {ref: PostgreSQL.profiles.analytical-ssd}}
```

Figure 2: Input contract (excerpt): one catalog, two axes. Workload entries state *why*/*when*/non-scope down to query-level intent and declare which systems they support; system entries span a spectrum from a managed endpoint to a fully tunable deployment. Profiles derive knob values from the experiment's resource limits and declare their preconditions — so the confounders of a fair comparison, tuning parity above all, become explicit and checkable rather than tacit.

reader can trace, cannot silently become an answer to an easier one. The summary's status field is the seed the contract generalizes. The output contract is the schema; the answer is the instance it governs, mirroring how the input contract governs the specification. Because the answer is grounded this way, the reader need not be a program: a person can interrogate a completed study and receive claims anchored in the recorded run. Conversational analytics operates over given data [20]; here the object is a self-describing *experiment*, which makes the exchange auditable rather than merely fluent. As elsewhere in the design, the bound is on form, not truth: it cannot make a conclusion

```yaml
# experiment.yml -- written by the agent,
# validated before touching the cluster
title: join performance under concurrency
hypothesis: pg_duckdb's join edge narrows
  under concurrency; rival: memory
  starvation, not thread contention
discriminates: [system, concurrency]
workload:
  name: tpch
  params: {active_queries: [5,7,8,9,21]}
  rounds: [1, 4]            # client sweep
systems:
  - {name: PostgreSQL, profile: analytical-ssd}
  - {name: PgDuckDB,  profile: analytical-ssd}
resources:
  memory: [{limit: 32Gi}, {limit: 64Gi}]
# one shared profile = tuning parity, stated
```

Figure 3: The agent's experiment specification: declarative, archived with the results, and stating before the run the hypothesis and the rival its factors are chosen to separate.

```
## Connection PgDuckDB-1-2-1
node: cl-node-07  (requested: benchmark)
details: report/execution.md
monitoring: report/monitoring.md

## Tests
[ok]   workflow as planned
[ok]   no SUT container restarts
[ok]   no SQL errors or result mismatch
[skip] monitoring: phase under one
       scrape interval (not a failure)
[warn] Q21 3.4x the PostgreSQL baseline
       within this experiment (312 s)
       -> logs/driver-PgDuckDB-1-2-1.log

## Provenance
spec: experiment.yml  (submitted copy)
images: postgres:18.3, pgduckdb:18-v1.1.1
        (tags, not digests)
cross-experiment comparison: not checked
  -- the agent must scope it itself
```

Figure 4: Output contract (excerpt of a generated summary): every section links into the result folder, and within-experiment validity tests precede any metric. What the contract cannot check — here, comparison across experiments — it names as the agent's responsibility rather than omitting.

correct, but it keeps each claim standing next to the evidence that must support it.

Result contract. The generated summary lets a bounded-context reader move from aggregate to evidence without human guidance: it is the declared entry point, with tiered file groups and links into the folder, and validity tests precede any metric (Figure 4). These tests are within-experiment — workflow ran as planned, no container restarted, no result-set mismatch across systems. Comparison *across* experiments the contract deliberately does not adjudicate: scoping it is a judgment the agent must make, a boundary the contract states rather than hides, alongside the other limits it records for itself, such as image tags carrying no content digest.

From numbers to explanations. What distinguishes an experimenter from a measurement device is explaining effects, not only obtaining numbers, and the contract carries explanation at three layers: within-experiment attribution over joinable metrics

and monitoring data, hypothesis-discriminating follow-up experiments in the loop, and the hypothesis field that turns a trajectory into a documented investigation. Some explanations bottom out below benchmarking's instruments; the agent explains to the resolution its instrument provides and reports the residual as open, as a careful human does.

4 Prototype and Demonstration

We realize the contract on top of Bexhoma, a Kubernetes-native orchestrator for benchmark experiments on data systems [10, 11]. It deploys systems under test, loaders, drivers, and monitoring as cluster workloads, and its deployment configuration is expressive enough to treat placement, storage class, resource limits, and system configuration as experimental factors rather than fixed settings. Being cloud-native, it records what it does: metrics per component and phase, logs of every deployment step, planned versus actual workflow, and the full configuration of each run are collected into one result folder per experiment. That instinct — log everything, keep it together, keep it reproducible — is what makes the output side of the contract a matter of *describing* what is already there rather than instrumenting what is missing.

The prototype's running example is a single user task:

> *Is pg_duckdb better on joins than PostgreSQL, even under concurrency? I have a 10 GB dataset, and our servers have at most 64 GB of RAM.*

A deliberately simpler question than the one we opened with — one instrument family, two systems, one cluster — but not a textbook comparison: vectorized execution that wins a single query stream is also the design most exposed when streams contend for threads and the memory budget per stream shrinks, so the outcome is open. It exercises every artifact in sequence: the catalog's intent fields rule out YCSB (non-scope: joins) and select TPC-H with a join-heavy subset, swept across concurrent client rounds (Figure 2); 10 GB maps to scale factor 10 and the memory budget to resource limits validated against the environment descriptor; the specification declares system, round count, and memory limit as factors; and the result contract carries per-query attribution over joined monitoring data. The two factors also discriminate between rival explanations within a single experiment: if concurrent degradation is memory starvation, raising the limit recovers it; if it is thread contention, it does not. Where evidence still underdetermines the cause, the loop continues with a follow-up — a storage-class variation, or a hardware micro-benchmark against the same volume. Throughout, the prototype plays the role proper to a vision paper's evidence: it is not the vision realized, but the argument that the vision is testable — and near.

5 Research Agenda

Cheap experiments are arriving whether or not we specify them well; what this agenda decides is whether the result is trustworthy or merely abundant. The danger is not new — selective reporting, unreplicable margins, and results too favourable to their author long predate language models, which is why this community built audited benchmarks and wrote down its pitfalls [13, 30] — but the speed is, and the governance we inherited assumes a human bottleneck that is about to disappear. Two questions organize what remains: how an automated experimenter can be a trustworthy one, and what becomes askable once it is — trust and reach. Neither is a matter of building the system; the engineering belongs to the prototype, and these questions outlast it. They are also coupled: reach without trust yields evidence nobody can believe, and trust without reach is the careful, occasional benchmarking we already practise.

5.1 A Methodology for Non-Human Experimenters

> *How does an automated experimenter remain an honest one, and how much of an inquiry must be reproducible: the experiment, or the path that led to it?*

5.2 What Becomes Possible

> *What can be asked once designing, running, and interpreting an experiment is cheap?*

6 Conclusion

Benchmarking automated its execution years ago; this paper argues for automating its experimenter. The beneficiaries are not only human: research groups spend their first weeks on mechanics rather than questions, release gates report *why* a query regressed rather than merely that one did, and reviewers re-run a study from its archived specification instead of trusting a badge. Machines join them, as the tuning and diagnosis agents that today rebuild bespoke harnesses become *clients* of a shared one — a service whose consumers are other services is what AI-readiness means taken seriously.

The contract serves these users by supplying what each lacks. An engineer declares a few systems and receives results rigorous by construction, the factorial design run, repetitions taken, variance reported, artifacts flagged; a scientist brings a workload no benchmark encodes and receives freedom from the labor of testbeds and multi-dimensional sweeps. These are the ends of one spectrum, along which the human brings more and the tool supplies less, from declaring a system to inventing a benchmark. What stays constant is the guarantee and its limit: the contract *ensures* the mechanizable best practices and *makes explicit* the judgements it cannot mechanize, whether a workload is representative, whether a comparison is fair. It relieves the human of the bookkeeping, not of the science. One friction remains: several commercial licenses restrict publishing benchmark results, and benchmarking at near-zero marginal cost collides with that tradition, a tension the community must negotiate rather than the contract resolve.

The enabling step is small: a two-sided, enforced contract — an intent-bearing catalog of workloads and systems in, navigable and validity-first results out — that turns benchmark studies from human-triggered campaigns into a standing capability. Binding the experimenter to a contract also confines its nondeterminism to design time: the study it produces is as replayable as any human-authored one, and more strictly specified. Our thesis, made testable by the prototype and its planned ablations, is that AI-readiness is a property of interfaces, not of intelligence. If it holds, we expect the benchmark papers of the early 2030s to ship with an agent-runnable specification — and the experimenter itself to become a citable, replayable artifact.

Acknowledgments

References

[1] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proceedings of the 2017 International Conference on Management of Data (SIGMOD)*. ACM, 1009–1024. doi:10.1145/3035918.3064029

[2] Mubashara Akhtar, Omar Benjelloun, Costanza Conforti, Pieter Gijsbers, Joan Giner-Miguelez, Nitisha Jain, Michael Kuchnik, Quentin Lhoest, Pierre Marcenac, Manil Maskey, et al. 2024. Croissant: A Metadata Format for ML-Ready Datasets. In *Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning (DEEM@SIGMOD)*. ACM, 1–6. doi:10.1145/3650203.3663326

[3] Peter Boncz. 2022. Technical Perspective: The 'Art' of Automatic Benchmark Extraction. *Commun. ACM* 65, 12 (2022), 104. doi:10.1145/3567465

[4] Peter Boncz, Thomas Neumann, and Orri Erling. 2014. TPC-H Analyzed: Hidden Messages and Lessons Learned from an Influential Benchmark. In *Performance Evaluation and Benchmarking (TPCTC 2013) (Lecture Notes in Computer Science, Vol. 8391)*. Springer, 61–76. doi:10.1007/978-3-319-04936-6_5

[5] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. 2025. AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 7.

[6] Sunyanan Choochotkaew, Tatsuhiro Chiba, Scott Trent, Takeshi Yoshimura, and Marcelo Amaral. 2022. AutoDECK: Automated Declarative Performance Evaluation and Tuning Framework on Kubernetes. In *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*. IEEE, 309–314. doi:10.1109/CLOUD55607.2022.00052

[7] Shaleen Deep, Anja Gruenheid, Kruthi Nagaraj, Hiro Naito, Jeff Naughton, and Stratis Viglas. 2022. DIAMetrics: Benchmarking Query Engines at Scale. *Commun. ACM* 65, 12 (2022), 105–112. doi:10.1145/3565633

[8] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudré-Mauroux. 2013. OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases. *Proceedings of the VLDB Endowment* 7, 4 (2013), 277–288. doi:10.14778/2732240.2732246

[9] Patrick K. Erdelt. 2021. A Framework for Supporting Repetition and Evaluation in the Process of Cloud-Based DBMS Performance Benchmarking. In *Performance Evaluation and Benchmarking (TPCTC 2020) (Lecture Notes in Computer Science, Vol. 12752)*. Springer, 75–92. doi:10.1007/978-3-030-84924-5_6

[10] Patrick K. Erdelt. 2022. Orchestrating DBMS Benchmarking in the Cloud with Kubernetes. In *Performance Evaluation and Benchmarking (TPCTC 2021) (Lecture Notes in Computer Science, Vol. 13169)*. Springer, 81–97. doi:10.1007/978-3-030-94437-7_6

[11] Patrick K. Erdelt. 2024. A Cloud-Native Adoption of Classical DBMS Performance Benchmarks and Tools. In *Performance Evaluation and Benchmarking (TPCTC 2023) (Lecture Notes in Computer Science, Vol. 14247)*. Springer, 124–142. doi:10.1007/978-3-031-68031-1_9

[12] Patrick K. Erdelt and Jascha Jestel. 2022. DBMS-Benchmarker: Benchmark and Evaluate DBMS in Python. *Journal of Open Source Software* 7, 79 (2022), 4628. doi:10.21105/joss.04628

[13] Michael Fruth, Stefanie Scherzinger, Wolfgang Mauerer, and Ralf Ramsauer. 2022. Tell-Tale Tail Latencies: Pitfalls and Perils in Database Benchmarking. In *Performance Evaluation and Benchmarking (TPCTC 2021) (Lecture Notes in Computer Science, Vol. 13169)*. Springer, 119–134. doi:10.1007/978-3-030-94437-7_8

[14] Victor Giannakouris and Immanuel Trummer. 2025. λ-Tune: Harnessing Large Language Models for Automated Database System Tuning. *Proceedings of the ACM on Management of Data (SIGMOD)* 3, 1 (2025). doi:10.1145/3709652

[15] Sören Henning and Wilhelm Hasselbring. 2021. Theodolite: Scalability Benchmarking of Distributed Stream Processing Engines in Microservice Architectures. *Big Data Research* 25 (2021), 100209. doi:10.1016/j.bdr.2021.100209

[16] Xinmei Huang, Haoyang Li, Jing Zhang, Xinxin Zhao, Zhiming Yao, Yiyan Li, Tieying Zhang, Jianjun Chen, Hong Chen, and Cuiping Li. 2025. E2ETune: End-to-End Knob Tuning via Fine-Tuned Generative Language Model. *Proceedings of the VLDB Endowment* 18, 13 (2025), 5540–5554. doi:10.14778/3773731.3773732

[17] Saurabh Jha, Rohan Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya Bhavya, Mudit Verma, Harshit Kumar, et al. 2025. ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. arXiv preprint arXiv:2502.05352.

[18] Michiel Van Kenhove, Merlijn Sebrechts, Filip De Turck, and Bruno Volckaert. 2023. Cloud-Native-Bench: an Extensible Benchmarking Framework to Streamline Cloud Performance Tests. In *2023 IEEE 12th International Conference on Cloud Networking (CloudNet)*. IEEE, 86–93. doi:10.1109/CloudNet59005.2023.10490079

[19] Charalampos Kostopoulos, George Mouchakis, Antonis Troumpoukis, Nefeli Prokopaki-Kostopoulou, Angelos Charalambidis, and Stasinos Konstantopoulos. 2021. KOBE: Cloud-Native Open Benchmarking Engine for Federated Query Processors. In *The Semantic Web (ESWC 2021) (Lecture Notes in Computer Science, Vol. 12731)*. Springer, 664–679. doi:10.1007/978-3-030-77385-4_40

[20] Eugenie Lai et al. 2025. KramaBench: A Benchmark for AI Systems on Data-to-Insight Pipelines over Data Lakes. arXiv preprint arXiv:2506.06541.

[21] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2024. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1939–1952. doi:10.14778/3659437.3659449

[22] Yiyan Li, Haoyang Li, Jing Zhang, Renata Borovica-Gajic, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, Cuiping Li, and Hong Chen. 2025. AgentTune: An Agent-Based Large Language Model Framework for Database Knob Tuning. *Proceedings of the ACM on Management of Data (SIGMOD)* 3, 6 (2025). doi:10.1145/3769758 Article 293.

[23] Wan Shen Lim, Matthew Butrovich, William Zhang, Andrew Crotty, Lin Ma, Peijing Xu, Johannes Gehrke, and Andrew Pavlo. 2023. Database Gyms. In *Conference on Innovative Data Systems Research (CIDR)*.

[24] Qi Lin, Zhenyu Zhang, Viraj Thakkar, Zhenjie Sun, Mai Zheng, and Zhichao Cao. 2025. StorageXTuner: An LLM Agent-Driven Automatic Tuning Framework for Heterogeneous Storage Systems. arXiv preprint arXiv:2510.25017.

[25] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv preprint arXiv:2408.06292.

[26] MLCommons Science Working Group. 2025. An MLCommons Scientific Benchmarks Ontology. arXiv preprint arXiv:2511.05614.

[27] Moisés Silva-Mu noz, Gonzalo Calderon, Alberto Franzin, and Hugues Bersini. 2021. Determining a Consistent Experimental Setup for Benchmarking and Optimizing Databases. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '21 Companion)*. ACM, 1614–1621. doi:10.1145/3449726.3463180

[28] Abdelkader Ouared, Moussa Amrani, Abdelhak Chadli, and Pierre-Yves Schobbens. 2024. Deep Variability Modeling to Enhance Reproducibility of Database Performance Testing. *Cluster Computing* 27, 8 (2024), 11683–11708. doi:10.1007/s10586-024-04533-0

[29] Alessandro Vittorio Papadopoulos, Laurens Versluis, André Bauer, Nikolas Herbst, Jóakim von Kistowski, Ahmed Ali-Eldin, Cristina L. Abad, José Nelson Amaral, Petr Tůma, and Alexandru Iosup. 2021. Methodological Principles for Reproducible Performance Evaluation in Cloud Computing. *IEEE Transactions on Software Engineering* 47, 8 (2021), 1528–1543. doi:10.1109/TSE.2019.2927908

[30] Mark Raasveldt, Pedro Holanda, Tim Gubner, and Hannes Mühleisen. 2018. Fair Benchmarking Considered Difficult: Common Pitfalls in Database Performance Testing. In *Proceedings of the Workshop on Testing Database Systems (DBTest)*. ACM, 2:1–2:6. doi:10.1145/3209950.3209955

[31] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Zicheng Liu, and Emad Barsoum. 2025. Agent Laboratory: Using LLM Agents as Research Assistants. arXiv preprint arXiv:2501.04227.

[32] Daniel Seybold and Jörg Domaschka. 2021. Automated Benchmarking of DBMS with benchANT. In *Proceedings of the Conference on Innovative Data Systems and Applications (poster) (CEUR Workshop Proceedings, Vol. 3043)*.

[33] Daniel Seybold, Moritz Keppler, Daniel Gründler, and Jörg Domaschka. 2019. Mowgli: Finding Your Way in the DBMS Jungle. In *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering (ICPE)*. ACM, 321–332. doi:10.1145/3297663.3310303

[34] Toni Taipalus. 2024. Database Management System Performance Comparisons: A Systematic Literature Review. *Journal of Systems and Software* 208 (2024), 111872. doi:10.1016/j.jss.2023.111872

[35] Immanuel Trummer. 2022. DB-BERT: A Database Tuning Tool that "Reads the Manual". In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*. ACM, 190–203. doi:10.1145/3514221.3517843

[36] Johannes Wehrstein, Timo Eckmann, Matthias Jasny, and Carsten Binnig. 2026. Bespoke OLAP: Synthesizing Workload-Specific One-size-fits-one Database Engines. arXiv preprint arXiv:2603.02001.

[37] Johannes Wehrstein, Anton Winter, Timo Eckmann, and Carsten Binnig. 2026. Bespoke-Card: Why Tune When You Can Generate? Synthesizing Workload-Specific Cardinality Estimators. arXiv preprint arXiv:2606.09361.

[38] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, et al. 2016. The FAIR Guiding Principles for Scientific Data Management and Stewardship. *Scientific Data* 3 (2016), 160018. doi:10.1038/sdata.2016.18

[39] Christopher Zanoli, Andrea Giovannini, Tengjun Jin, Ana Klimovic, and Yotam Perlitz. 2026. ELT-Bench-Verified: Benchmark Quality Issues Underestimate AI Agent Capabilities. arXiv preprint arXiv:2603.29399.

[40] Xinyi Zhang, Tiantian Chen, Zhentao Han, Zhaoyan Hong, Wei Lu, Sheng Wang, Mo Sha, Anni Wang, Shuang Liu, Yakun Zhang, Feifei Li, and Xiaoyong Du. 2026. Why Database Manuals are Not Enough: Efficient and Reliable Configuration Tuning for DBMSs via Code-Driven LLM Agents. *Proceedings of the VLDB Endowment* 19, 6 (2026), 1358–1371. doi:10.14778/3797919.3797940

[41] Wei Zhou, Yuyang Gao, Xuanhe Zhou, Guoliang Li, et al. 2025. GaussMaster: An LLM-based Database Copilot System. *Proceedings of the VLDB Endowment* 18 (2025). arXiv:2506.23322.

[42] Xuanhe Zhou, Guoliang Li, Zhaoyan Sun, Zhiyuan Liu, Weize Chen, Jianming Wu, Jiesi Liu, Ruohang Feng, and Guoyang Zeng. 2024. D-Bot: Database Diagnosis System using Large Language Models. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2514–2527. doi:10.14778/3675034.3675043

[43] Linwu Zhu, Liqiang Gao, Yan Chen, Dan Zhu, and Jian Huang. 2026. LAP: An Agent-to-Instrument Protocol for Autonomous Science. arXiv preprint arXiv:2606.03755.